

Backend design and database architecture

A first draft of how the MVP backend could be put together: the components, how each major use case travels from input to output, and how the database would be laid out. This design is the main output so far. Written to be discussed and changed before real implementation starts.

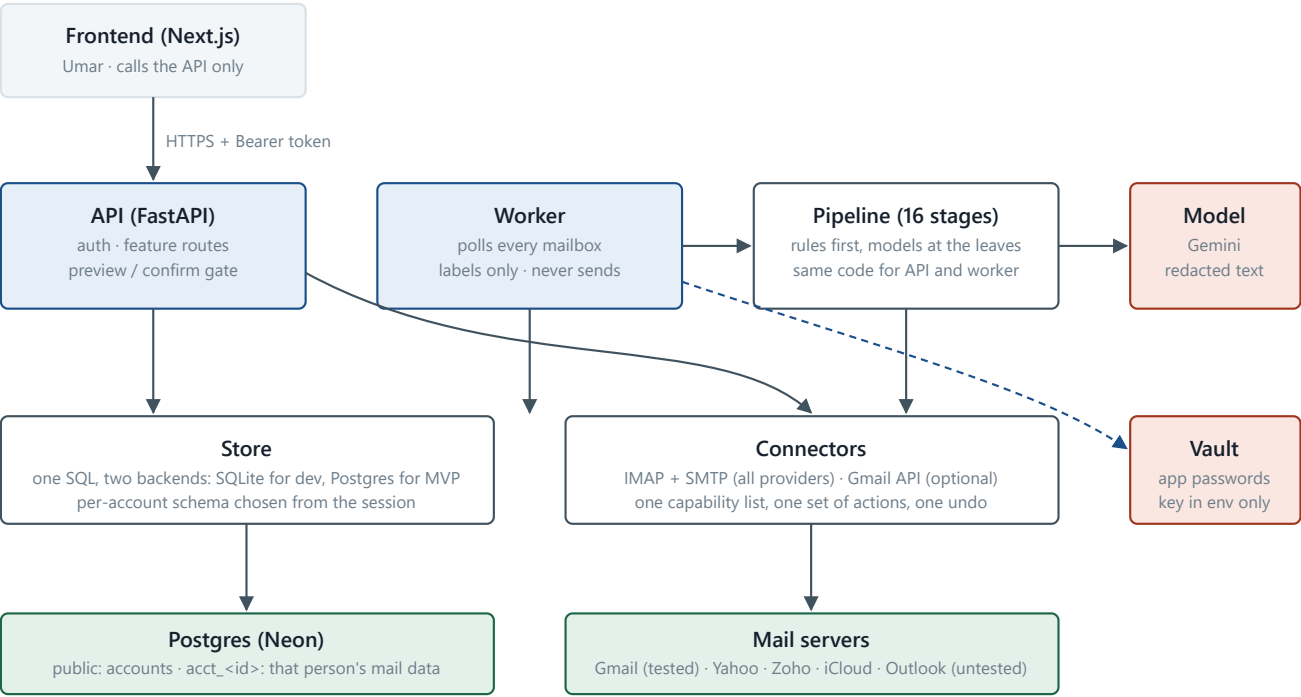
draft 0.1 · 19 September 2026 · Yahya Mateen · status: for review · implementation not started beyond exploratory checks

1 · What the backend has to do

The MVP is a web app where a person connects one or more mailboxes and Custodian reads, sorts and labels the mail, shows what needs attention, and lets the person act on it. The backend owns everything except the screens.

REQUIREMENT	WHERE IT COMES FROM
Accounts	Sign up, sign in, reset. A session reaches an account, never a mailbox on its own (UC-01, UC-02).
Mailboxes	Connect a mailbox by app password or provider sign-in; disconnect cleanly (UC-04 to UC-13). Target providers: Gmail (tested), Yahoo, Zoho and iCloud (standard IMAP, expected to work, not yet tried), Outlook (needs a Microsoft app registration and sign-in flow, not yet tried).
Always reading	New mail is noticed within seconds and passed through the pipeline without anyone at a keyboard.
Features	The 50 use cases: today's list, requests and promises, the pile, brands, unsubscribe, storage, spam rescue, reminders, rules, people, threads, search, digest, voice, calendar.
Safety	Nothing irreversible without the person seeing the exact wording and saying yes. Nothing is ever deleted. Credentials never in plain text. Model calls never see real names.
Separation	One customer's mail can never be read by a query meant for another customer.

2 · Components

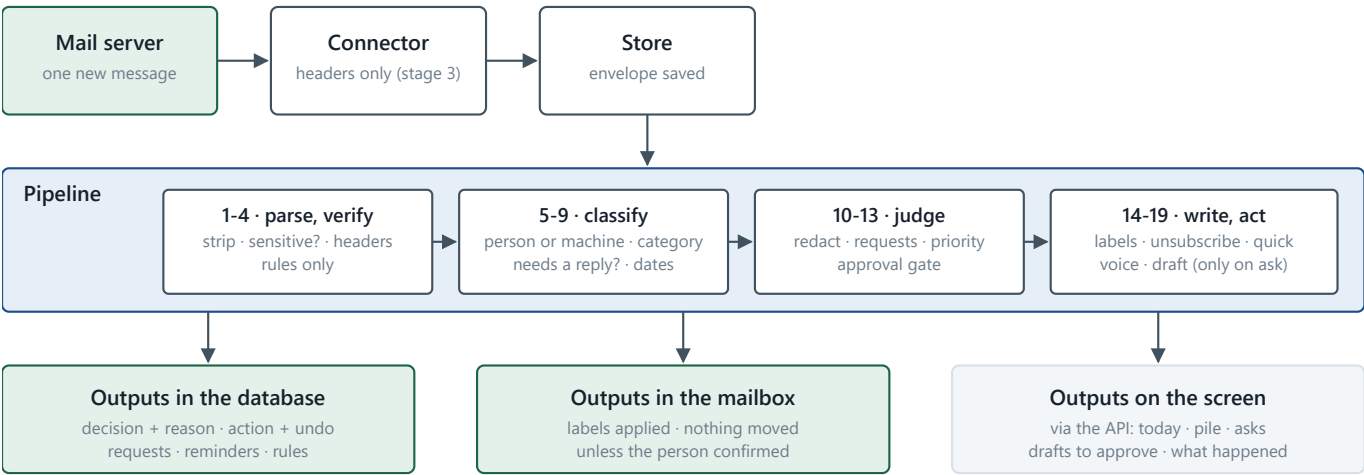


Solid lines are calls. Dashed: the worker reads each mailbox's app password from the vault; the API writes it there once at connect time. The API and the worker never call each other; they meet only in the database.

COMPONENT	RESPONSIBILITY	STATUS
API	Everything the frontend needs, as JSON. Checks the session, picks the account's store, calls the same functions the terminal scripts call. Holds the preview/confirm gate.	exploratory
Auth	Script password hashes, single-use confirm and reset tokens, session tokens, throttling on failed logins, one error message for wrong email or wrong password.	exploratory
Worker	One process, no browser. New mail first, a little backlog when idle, hourly housekeeping (requests scan, unsubscribe outcomes, digest). Cannot send, cannot clear.	exploratory
Pipeline	16 deterministic stages; models only where rules cannot decide. Unchanged from the POC. Redaction before every model call, restoration after.	POC code · to be reviewed
Store	One SQL dialect written once. SQLite for local work, Postgres for the MVP through a thin translation layer. Caches the handful of lookups that repeat per page.	exploratory
Connectors	IMAP/SMTP for every provider, Gmail API as an optional door. Same method names, same undo for each action, capability flags so callers never guess.	POC code · to be reviewed
Vault	App passwords encrypted with a key that exists only in the server's environment. Verified against the mail server before being stored.	exploratory

Outbound email	Confirmation and reset emails to the account holder. No service chosen yet; tokens are currently returned to the caller for testing.	not started
Chat	Sentence in, plain answer out, over the same feature functions. Design not started; only the typed-rules box exists.	not started
Deployment	API and worker hosted beside the database. Nothing is deployed; everything runs on one laptop.	not started

3 • Every message, end to end



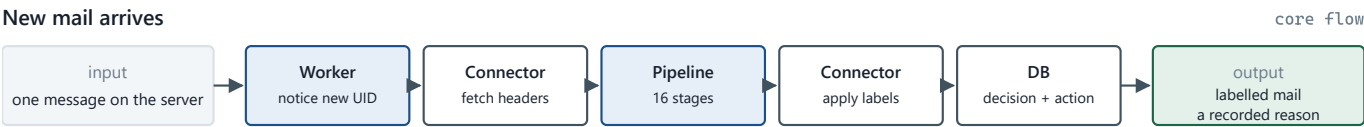
Every message takes this path. Earlier stages resolve most mail by rules alone; the model is only consulted where rules cannot decide, and only on redacted text.

4 • Major use cases, input to output

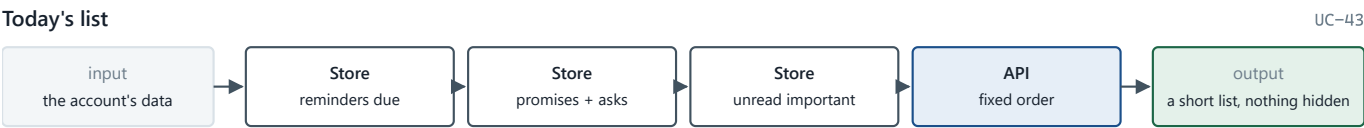
Each lane reads left to right: what comes in, which components touch it and in what order, what comes out. Red boxes are the gate; nothing to their right happens without the person's yes.



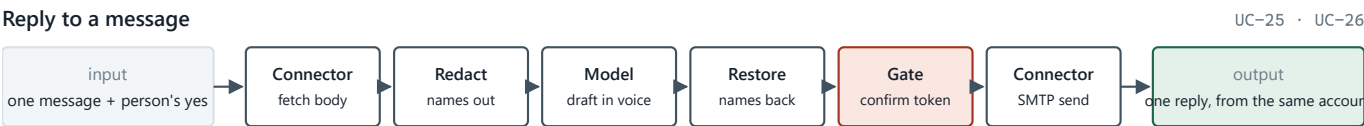
reversible: no approval needed, every step recorded with its undo



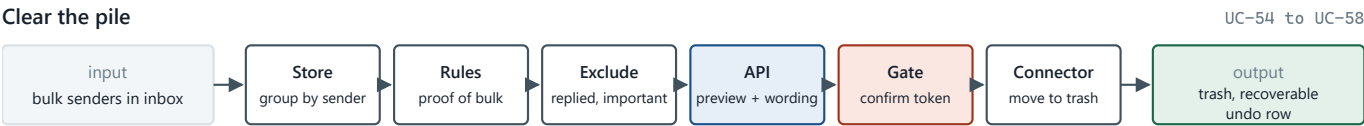
reversible: no approval needed, every step recorded with its undo



reversible: no approval needed, every step recorded with its undo



goes through the preview / confirm gate: refused without the person's yes



goes through the preview / confirm gate: refused without the person's yes

Unsubscribe

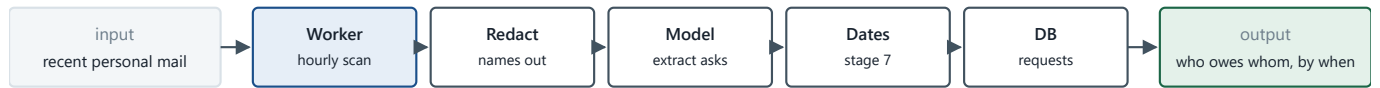
UC-27 · BR-66



goes through the preview / confirm gate: refused without the person's yes

Requests and promises

UC-33 · UC-45



reversible: no approval needed, every step recorded with its undo

Reminders and calendar

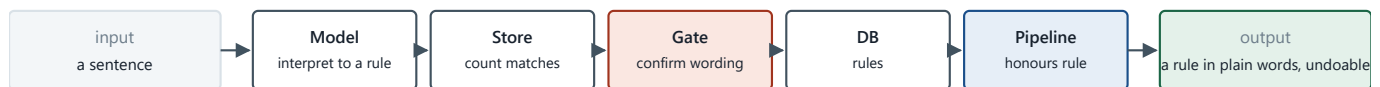
UC-30 · UC-46



reversible: no approval needed, every step recorded with its undo

Typed rules / important people

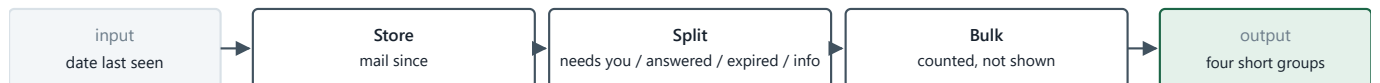
UC-38 · UC-44



goes through the preview / confirm gate: refused without the person's yes

Catch-up after time away

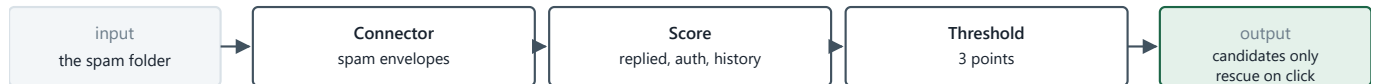
UC-42



reversible: no approval needed, every step recorded with its undo

Spam rescue

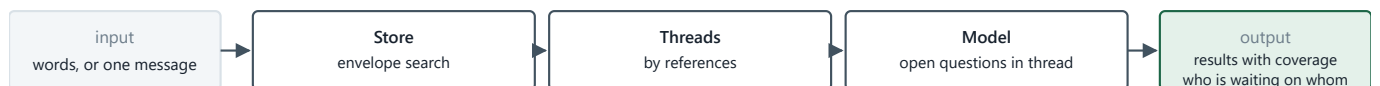
UC-31



reversible: no approval needed, every step recorded with its undo

Search and threads

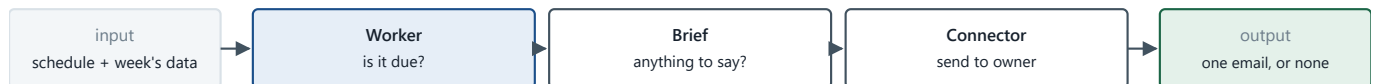
UC-28 · UC-40



reversible: no approval needed, every step recorded with its undo

Digest

UC-36



reversible: no approval needed, every step recorded with its undo

Disconnect a mailbox

UC-13

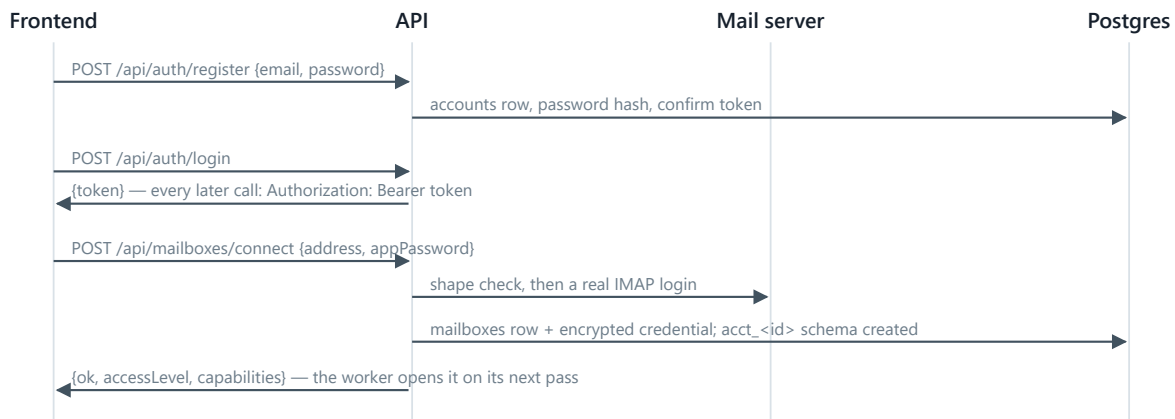


goes through the preview / confirm gate: refused without the person's yes

The account screens (UC-01 to UC-03, UC-47 to UC-49) are frontend flows over the auth routes and are not drawn here.

5 • How requests travel

5.1 • Sign in and connect a mailbox



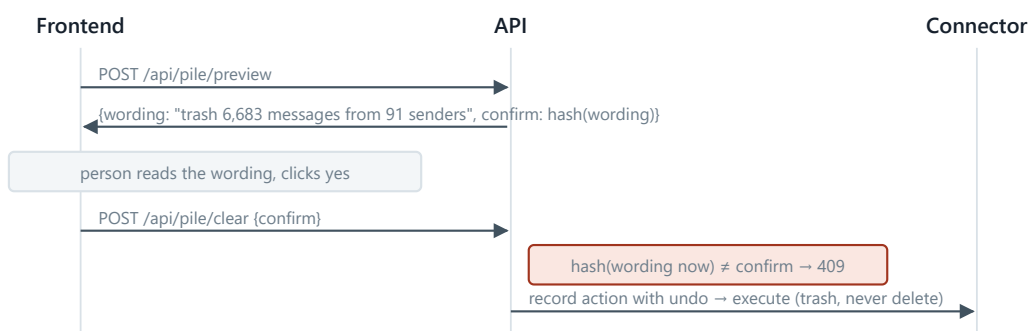
The password never goes to the log, never goes to the database in the clear, and is only stored after the mail server accepted it. If the server refuses, the person gets the provider's real reason, not a generic error.

5.2 • One pass of the worker

```
for each connected mailbox (own connection, own store)
  new = mailbox.new_since(bookmark)          ← highest UID (IMAP) or history id (Gmail)
  if new:
    for each message: save envelope → pipeline → labels applied → decision recorded
  else:
    scan 3 older messages                    ← backlog only when nothing new arrived
  once an hour:
    scan recent mail for requests and promises
    check whether unsubscribed senders actually stopped
    send the digest if it is due and there is something to say
sleep 20 s
```

The worker holds no approval, so it can only do reversible things. Everything it does is written to the actions table with how to undo it, before the call is made.

5.3 • An irreversible action

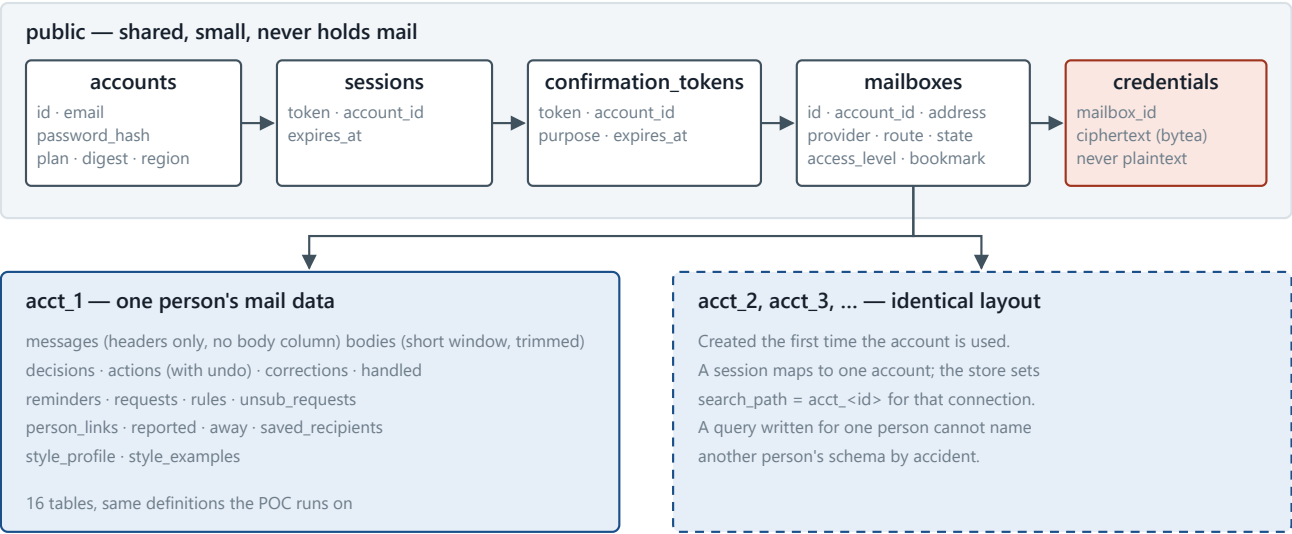


The token is bound to the wording. If the pile changed between preview and confirm, the wording changes, the token no longer fits, and the request is refused. This applies to clearing, unsubscribing, sending, batch

forwarding and creating rules. The same gate exists in the terminal version; the API reuses it rather than adding a second one.

6 • Database architecture

Two layers. A small shared layer that knows *who* the customers are, and one private schema per account that holds everything *about* that person's mail.



Why one schema per account

OPTION	HOW IT WORKS	WHY NOT / WHY YES
account_id column	All customers in the same tables, every query filtered by account_id.	Common and fast to set up. One forgotten WHERE exposes another customer's mail. Rejected for now because the privacy promise is the product.
One schema per account proposed	Same tables, one namespace per account. The connection is pointed at the account's schema once, from the session.	Separation is structural, not a habit. Every existing script keeps its SQL unchanged. Fine to several thousand accounts; would need revisiting far beyond that.
One database per account	Full physical separation.	Strongest separation, but connection counts and migrations become a problem early. Too heavy for an MVP.

Rules the schema enforces

- **No message body column.** Bodies live in a separate table, kept for a short window and trimmed on start. Headers are enough for everything except drafting.
- **Credentials in their own table,** encrypted before they arrive. A dump of any other table never contains one.
- **Every action row carries its undo** and is written before the call is made. If the process dies in between, the record errs toward "it may have happened".
- **Nothing is ever deleted from a mailbox.** Trash is a move; the provider keeps it recoverable.
- **Message-ID is the key,** not the provider's id, so the same mail seen through IMAP and Gmail API is one row.

7 • Where things run

PIECE	TODAY	PROPOSED FOR THE MVP
Database	Neon (hosted Postgres), US East. Measured from Pakistan: the same query took 1 s one moment and up to 39 s the next. Trimming the query helped; distance did not go away.	Neon project in Singapore or Frankfurt. Same schema, re-import takes minutes.
API + worker	One laptop.	One small server in the same region as the database, so the chatty part of the traffic stays inside one datacentre. The link to the mail servers is the only long hop.
Local development	SQLite file, same SQL, no network.	Unchanged. The store picks the backend from the environment.
Secrets	Environment variables on the laptop.	Environment on the server: database URL, model key, vault key. None in the repo, none in the database.

8 • Open questions for this review

1. **Region.** Singapore or Frankfurt for the database and server? Depends on where the first users are.
2. **Per-user mailbox connections in the API.** The worker opens each user's mailbox from the vault. The API currently reuses one connection opened at startup, which is fine for one user and wrong for two. The fix is known (open from the vault per request, cache per account); it should be agreed before it is built.
3. **Outbound email for confirmations and resets.** Which service, and from which address.
4. **The chat.** Agreed that there is one. Not designed. It should call the same feature functions the routes call, and pass through the same gate; whether it is a route that returns text, or something streaming, is open.
5. **Retention.** How long message bodies are kept, and whether the person can choose.
6. **Hosting.** Where the API and worker live, and who holds the environment secrets.

9 • What exists right now, plainly

This document is the main work product so far. To check that the design holds together, some exploratory code was written on one laptop against one test mailbox: the shared tables were applied to a hosted Postgres, a copy of the test mailbox was loaded, and the sign-in and a few of the routes were tried. That code is exploratory, not production quality, and is expected to be reshaped or replaced after this review. Nothing is deployed, nothing has been used by more than one person, and nothing has been pushed to a shared repository.